Retrieval-Augmented Generation

Why retrieval

A model only knows what was in its training data. Retrieval-augmented generation, or RAG, finds the passages in your own documents that relate to a question and gives them to the model together with the question, so the answer is grounded in sources you control and can be checked.

Extracting text from PDFs

Most PDFs carry a text layer that can be read directly, page by page, along with the font size of each line. Scanned PDFs are only images of pages and need optical character recognition before any text can be indexed.

Clean the text before chunking. Running headers, footers and page numbers repeat on every page and add noise to every chunk, so detect and remove them. Rejoin words that were hyphenated at the end of a line.

Chunking documents

Retrieval works on chunks, not whole documents. A chunk of 300 to 500 tokens is small enough to be specific and large enough to carry context. Follow the document's structure: start a new chunk at a major heading instead of cutting a section in half.

Add an overlap of about 10 to 20 percent between consecutive chunks, so a fact that spans a boundary can be found from either side. Keep the page number and the section heading of every chunk; you will need them for citations.

Embeddings

An embedding model turns text into a vector of numbers, so that texts with similar meaning end up close together. Similarity is usually measured with cosine similarity.

Questions and documents must be embedded with the same model. Vectors from different models live in different spaces and cannot be compared, so changing the embedding model means re-embedding every chunk.

Vector databases

A vector database stores embeddings and finds the nearest ones to a query vector. Exact search compares the query with every vector, which becomes slow with millions of chunks, so most systems use approximate nearest neighbour indexes such as HNSW. PostgreSQL can do this with the pgvector extension, which keeps vectors next to the rest of the application data.

Hybrid search

Vector search finds passages with the same meaning even when the wording differs, but it can miss exact terms such as product names, error codes or acronyms. Keyword search catches those exact terms but misses paraphrases.

Hybrid search runs both and merges the two ranked lists. Reciprocal rank fusion is a simple and robust way to merge them: each passage scores one divided by the sum of a constant k and its rank in each list, with k commonly set to 60. Because it uses only ranks, the two retrievers' scores never need to be on the same scale.

Citing sources

Number the passages in the prompt and ask the model to cite the number after each claim. Then check the citations in code: drop any number that does not match a passage, and show each cited passage with its document and page so the reader can verify the answer.

When no passage is relevant enough, say that the documents do not cover the question instead of letting the model answer from memory.

Evaluating retrieval

Build a test set of questions, each labelled with the passage that answers it. Measure recall at k, the share of questions whose answer appears in the top k results, and mean reciprocal rank, which rewards putting the right passage first. Include questions the documents cannot answer, to check that the system declines them. Run the evaluation after every change to chunking, embeddings or ranking.